

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 - Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 - Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	Sumetha Sai S	Reg. No.:	192521097
Section / Batch:		Date:	20/07/2026

Code of ConductI, Sumetha Sai S / 192521097 (Name / Reg No.)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: [Signature]**Instructions**

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:		100 Marks	

			➤ Convert the infix expression into postfix notation. ➤ Show the operator stack after every step. ➤ Evaluate the postfix expression using a stack. ➤ State the final result.
31	192525185	SHAIK ROSHNI	1. Ten documents are requested for printing. Illustrate the mechanism of printer operations. Consider the document number as 1003, 1008, 1010, 1005, 1009, 1004. Identify the suitable data structure and draw the diagram to print the documents. 2. The university announced the selected candidates register number for placement training. The student XXX, Register Number 20142010 wishes to check whether his name is listed or not. The list is not sorted in any order. Identify the searching technique that can be applied and explain the searching steps with suitable procedure. List includes 20142015, 20142033, 20142011, 20142017, 20142010, 20142056, 20142003
32	192525214	SOPPA JASWANTH	1. Insert the following elements 150, 250, 350, 450, 550 into the double linked list and delete the elements 150 and 250 from the double linked list. Show the linked structure using a node diagram before and after delete operations. 2. A circular queue has a size of 6. Execute the following operations: Enqueue(10), Enqueue(20), Enqueue(30), Dequeue(), Enqueue(40), Enqueue(50), Enqueue(60), Enqueue(70). Draw the queue after each operation and identify the values of the front and rear pointers.
33	192524023	SREE B	1. Convert the expression $((a + b) + c * (d + e) + f) * (g + h)$ to 1) Prefix expression, 2) Postfix expression. 2. Perform the binary search in an array of n elements using a recursion function, in which a function has to call by itself. Use these elements given in sorted order and assume 88 as the key element. [23, 33, 43, 48, 53, 58, 61, 64, 68, 73, 88]
34	192521097	SUMETHA SRI S	1. How a queue may be maintained by a circular array with MAXSIZE = 6. Observe that queue always occupies consecutive locations except when it occupies locations at the beginning and at the end of the array. If the queue is viewed as a circular array, this means that it still occupies consecutive locations. The queue is empty when count = 0; Perform the following Enqueue and Dequeue operations. a) Initially QUEUE is empty, b) A, B, C are Enqueued, c) A is Dequeued, d) D, E, F are Enqueued, e) B & C are Dequeued, f) G is Enqueued, g) D & E are Dequeued, h) H & I are Enqueued, i) F is dequeued, j) G & H are dequeued k) I is Dequeued. Demonstrate with a neat sketch of Front, Rear and count in a queue and circular queue.

38	192524251	VAISHNAV M	- Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses an explicit stack. Compare both approaches in terms of memory usage, execution flow, and risk of stack overflow. - A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: Push(A), Push(B), Push(C), Push(D), Push(E), Push(F). Identify the error and suggest two possible solutions.
39	192525187	VEERA SANTHOSH A	- Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. - A queue implemented using an array report "Queue Full" even though some positions are empty. - Analyze why this occurs. - Explain how a circular queue solves this problem.
40	192511226	VINIL KUMAR S	- Consider the singly linked list: 10 → 20 → 30 → 40 → 50. Delete the node containing 30. Explain how the links change after deletion. - Given two sorted linked lists: L1: 1 → 3 → 5 → 7 L2: 2 → 4 → 6 → 8

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge

Signature: _____

Date: _____

Course Coordinator

Signature: _____

Date: _____

Program Director

Signature: _____

Date: _____

CO2
ASSESSMENT TOOL-1

1. How a queue may be maintained by a circular array with $\text{MAXSIZE} = 6$. Observe that queue always occupies consecutive locations except when it occupies locations at the beginning and at the end of the array. If the queue is viewed as a circular array, this means that it still occupies consecutive locations. The queue is empty when $\text{count} = 0$; perform the following Enqueue and Dequeue operations.

a) Initially QUEUE is empty. b) A, B, C are Enqueued, c) A is dequeued, d) D, E, F are Enqueued, e) B & C are dequeued, f) G is Enqueued, g) D & E are dequeued, h) H and I are Enqueued, i) F is dequeued, j) G & H are dequeued.

Aim: To perform Enqueue and Dequeue operations on a circular Queue of size 6 and observe the changes in Front, Rear and Count.

Theory:

The queue is

- * Empty when $\text{Count} = 0$
- * Full when $\text{Count} = \text{MAXSIZE}$

Step-by-Step Execution:

Step (a): Initially Queue is Empty

Index:	0	1	2	3	4	5
Queue:	-	-	-	-	-	-

Front = 0 Rear = -1 Count = 0

Step (b): Enqueue A, B, C

Index:	0	1	2	3	4	5
Queue:	A	B	C	-	-	-

Front = 0 Rear = 2 Count = 3

Step (c): Dequeue A

Index:	0	1	2	3	4	5
Queue:	-	B	C	-	-	-

Front = 1 Rear = 2 Count = 2

Step (d): Enqueue D, E, F

Index:	0	1	2	3	4	5
Queue:	-	B	C	D	E	F

Front = 1 Rear = 5 Count = 5

Step (e): Dequeue B and C:

Index:	0	1	2	3	4	5
Queue:	-	-	-	D	E	F

Front = 3 Rear = 5 Count = 3

Step (f): Enqueue G

Index:	0	1	2	3	4	5
Queue:	G	-	-	D	E	F

Front = 3 Rear = 0 Count = 4

Step (g): Dequeue D and E

Index: 0 1 2 3 4 5

Queue: G - - - F

Front = 5

Rear = 0

Count = 2

Step (h) = Enqueue H and I

Index: 0 1 2 3 4 5

Queue: G H I - - F

Front = 5

Rear = 2

Count = 4

Step (i): Dequeue F:

Index: 0 1 2 3 4 5

Queue: G H I - - -

Front = 0

Rear = 2

Count = 3

Step (j): Dequeue G and H:

Index: 0 1 2 3 4 5

Queue: - - I - - -

Front = 2

Rear = 2

Count = 1

Step (k) = Dequeue I

Index: 0 1 2 3 4 5

Queue: - - - - -

Front = 3

Rear = 2

Count = 0

Summary Table:

Step	Operation	Front	Rear	Count
a	Empty	0	-1	0
b	Enqueue A, B, C	0	2	3
c	Dequeue A	1	2	3
d	Enqueue D, E, F	1	5	5
e	Dequeue B, C	3	5	3
f	Enqueue G	3	0	4
g	Dequeue D, E	5	0	2
h	Enqueue H, I	5	2	4
i	Dequeue F	0	2	3
j	Dequeue G, H	2	2	1
k	Dequeue I	3	2	0

- 2) Show the linked list configurations resulting from each series of list operations using the list ADT. Assume that lists L1 and L2 are empty at the beginning of each series. Show where the current position is in the list.
- (a) L1.append(10); L1.append(20); L1.append(15);
- (b) L2.append(10); L2.append(20); L2.append(15);
- L2.moveToStart(); L2.insert(39); L2.next(); L2.insert(12);

Aim:

To demonstrate the working of list ADT operations such as append, insert, moveToStart, and next using a linked list.

(a) Operations:

L1. append (10);

L1. append (20);

L1. append (15);

Step 1:

Append 10

Head



10 → NULL

Current Position → 10

~~Step 2:~~ Append 20

Head



10 → 20 → NULL

Current Position → 20

Step 3:

Append 15

Head



10 → 20 → 15 → NULL

Current Position → 15

Final:

Head



10 → 20 → 15 → NULL

Current Position → 15

(b) Operations :-

L2. append (10);

L2. append (20);

L2. append (15);

L2. move To Start ();

L2. insert (39);

L2. next ();

L2. next (12);

Step 1:

Append 10
10

Step 2: Append 20

10 → 20

Step 3: Append 15

10 → 20 → 15

Step 4: Move Current to Start

Current ↓

10 → 20 → 15

Step 5: Insert 39 before Current

Current

↓

39 → 10 → 20 → 15

Step 6:

Move Current to next

39 → 10 → 20 → 15

^

Current

Step 7: Insert 12 before Current

39 → 12 → 10 → 20 → 15 → NULL
 ^
 Current

Final: Head

39 → 12 → 10 → 20 → 15 → NULL
 ^
 Current position.

Result:

* - List L1: 10 → 20 → 15 → NULL

* - List L2: 39 → 12 → 10 → 20 → 15 → NULL

Unpaired